Data Science in the age of AI (part 3): Prompting is not building AI systems

Why the future of GenAI belongs to system architects, not prompt tweakers.

JOSE PARREÑO GARCIA
NOV 15, 2025

4 4 Share

This week I was reviewing a prototype of a recommendation system working purely based on LLMs. The idea was simple: can the LLM replace our Machine Learning personalisation and ranking model? I can't say it was a bad demo (but it wasn't the best either). But I can say I found a dozen holes in it when asking production grade systems.

It reminded me of what we talked about in Part 2, where anyone can now build a model, but building *a system that works when it matters* is a different story. This is the next step for focusing purely on generative AI applications.

The challenge is no longer "can we get the LLM to do something clever in the playground?" The challenge is "can we make it work reliably, at scale, inside the messy

reality of production?”

That is where prompt engineering ends and system design begins.

Senior Data Science Lead is a reader-supported publication. To receive new posts and support my work, consider becoming a free or paid subscriber.

Blog series

- ✅ Data Science in the age of AI (Part 1): The machine wrote my code (and that’s not the problem)
- ✅ Data Science in the age of AI (Part 2): If everyone can build a model, what are Data Scientists for?
- 📌 Data Science in the age of AI (part 3): Prompting is not building AI systems - ***(this post!)***
- Data Science in the age of AI (part 4): The science we can’t afford to lose - *(coming soon!)*
- Data Science in the age of AI (part 5): The future hybrid - *(coming soon!)*

What this blog will cover

- **Playground prompts vs production LLMs.** Why a clever prompt in a demo is not the same as a reliable feature in production, and how this mirrors the early days of AutoML.
- **Prompting as part of system design.** How real LLM systems chain prompts, inject context, integrate with other tools, and require deliberate orchestration.
- **LLM-specific design constraints.** What data scientists need to understand about latency, cost, context limits, and evaluation before shipping anything to production.
- **Reframing the data scientist's role (for those working purely with GenAI applications).** Why the skills you already use in feature engineering, pipeline design, and model evaluation transfer directly to building robust LLM workflows.
- **Closing thoughts.** Why prompting is plumbing, and why the impact comes from designing systems that are reliable, safe, and repeatable.

Let's begin!

Playground prompts vs production LLMs.

Prompting is not the product. And I have seen this story before.

When AutoML first appeared, many believed it would replace data scientists. 1 click and you had a model. But when those models went into production, or even when teams tried to improve offline predictive metrics… the cracks appeared. The hype faded when reality set in. And, this is why Data Scientists are still very much in demand nowadays.

Unfortunately, LLMs are now replaying the same cycle (but on steroids). It is easy to get something impressive in a playground:

- You copy a prompt from Twitter (yes, I can't seem to call it X).
- You get a coherent output.
- You show it in a demo.

But… just like training a model is not the same as running ML in production, prompting is not the same as designing an LLM system.

In a controlled setting, that can feel like the hard part is over. But the moment you try to use that same prompt inside a real product, the scope of the work changes completely. Now you need:

Version control and monitoring for prompt changes.

LLMs are stochastic systems, which means that even the same prompt will return different answers. In contrast, even though machine learning is predictive in its

output, it is deterministic in the sense that, given the same inputs, you get the same outputs. This means that LLMs are introducing an extra layer of complexity when it comes to control. Just imagine what would happen if prompts are tweaked without control or versioning!

Prompt evaluation pipelines to track quality over time.

If the LLM is replacing a predictive model, you can often reuse the same evaluation metrics. But for generative use cases, it is harder. A likely future is "LLM judges" that score prompts against predefined requirements (e.g. presence of guardrails, adherence to role constraints) and rate outputs for quality. Just as we monitor hyper parameters and tuned ML models for performance drift, we will need to monitor prompts and their output over time.

Output guardrails to catch bad or unsafe responses.

This can also be handled by another LLM, but in an adversarial role: actively probing the system for weaknesses or undesirable behaviours. This allows you to harden the system against edge cases before launch.

Failover logic when the LLM is slow or fails outright.

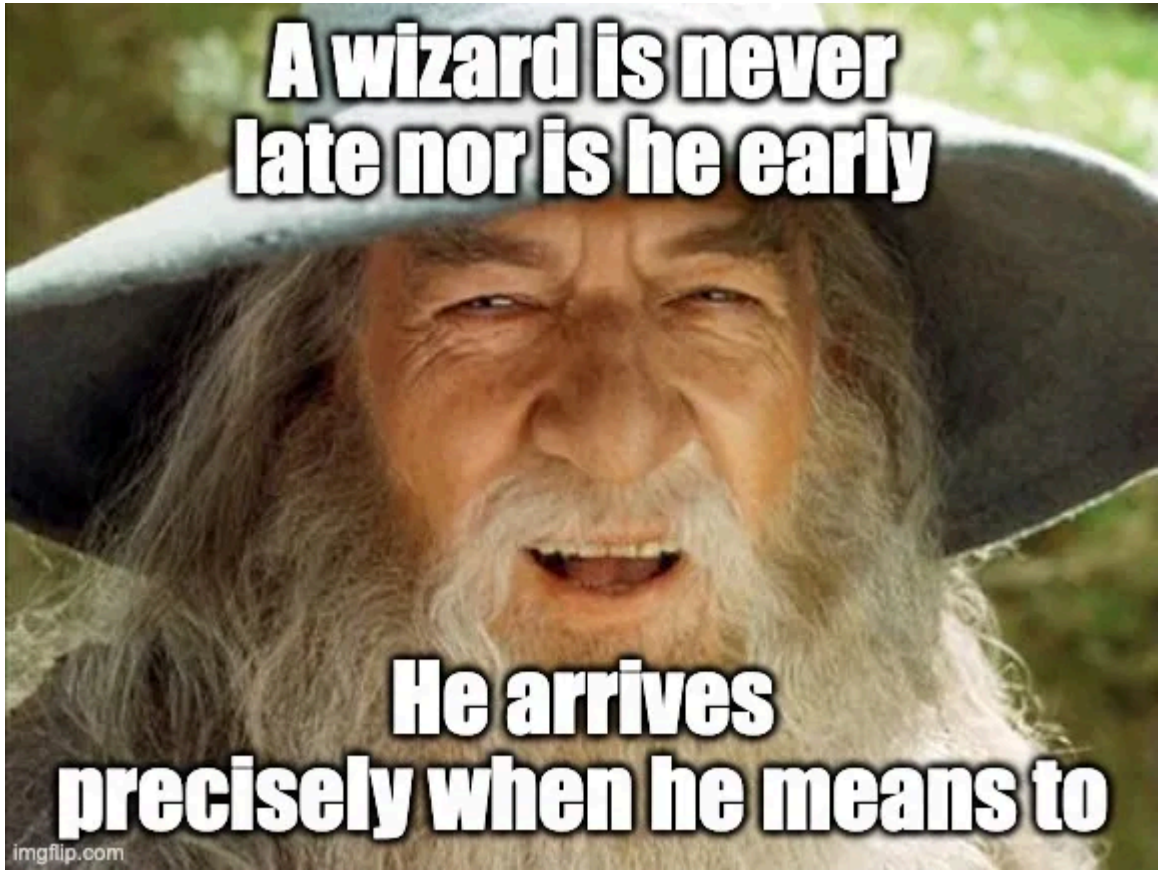
Relying on a single API endpoint is asking for trouble (remember how openAI flopped its GPT5 release by cancelling also GPT4?). Engineering systems are built on

redundancies. That might mean integrating with multiple providers, hosting your own model, or falling back to a simple heuristic. In ML, we often keep a fast, rules-based model as a backup. It is not optimal, but it is predictable and always available.

Even magic needs engineering

The magic that works in a demo still needs deployment specifications, quality assurance plans, and rollback strategies before it is ready for the real world. As Data Scientists, we have been dealing with these for a long time.

The risk is that companies fall in love with GenAI because it feels magical and forget that even magic needs engineering.



Prompting as part of system design.

In production, most working systems are a chain of steps that include prompts, logic, retrieval, and safeguards. If you have worked with tools like LangChain, or even n8n, you will have seen how quickly a “single prompt” turns into an orchestration layer.

Take a concrete example: football news scraping.

Old school scraping is a deterministic pipeline. You would crawl a site, parse the HTML with BeautifulSoup, extract article links, fetch the pages, and run NLP to classify and summarise them. It works, but it is brittle across layouts and expensive to maintain.

A modern, LLM-enabled flow keeps the reliable parts deterministic, but uses orchestrated prompts where structure is messy or intent matters:

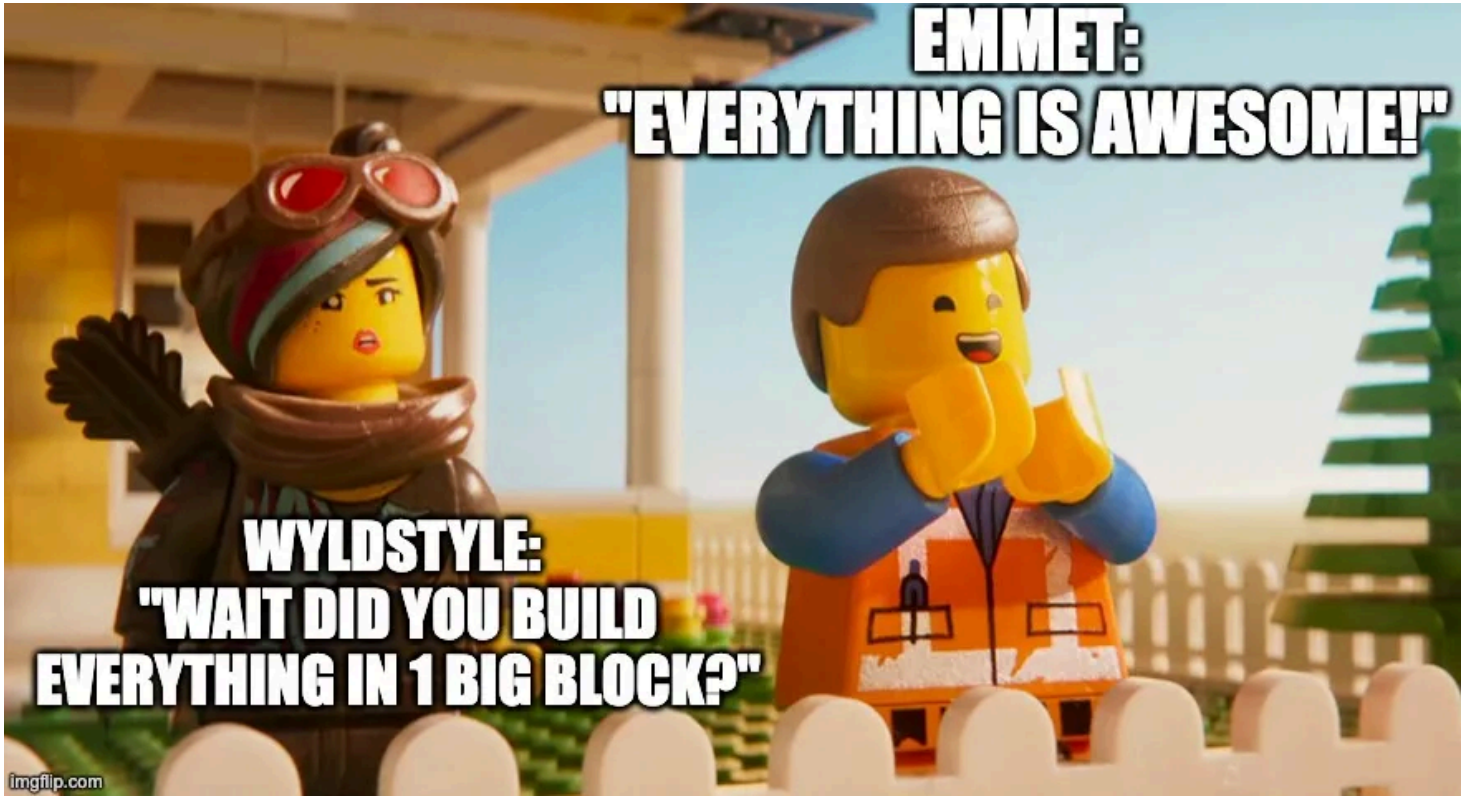
1. **Relevance filter**
   → Input: candidate articles with metadata.
   → Prompt: decide if the article is about *Team X* and return a JSON flag with confidence and rationale.
   → Guardrails: output schema validation and fallback keyword matching if confidence is low.
2. **Taxonomy classify**
   → Prompt: map each relevant article to one of `{upcoming_match, past_match_review, transfer, injury}`. Provide few-shot examples to pin the definitions.
   → Guardrails: reject "other" category unless confidence passes a threshold; low-confidence cases go to a second, cheaper classifier or human review.
3. **Structured summaries**
   → Prompt: produce machine-readable summaries in a strict template per category

(e.g. for an injury: `{ "player": "...", "injury_type": "...", "severity": "...", "expected_return": "..." }`).
→ Guardrails: low temperature, token cap, and schema checks. Retry once on invalid JSON, then fall back to deterministic regex snippets.

4. **Evaluation and routing**
   Run schema validation, banned phrase checks, and duplication checks automatically. For a small golden set, use LLM-as-evaluator to track precision and recall over time.
5. **Monitoring and versioning**
   Log category distributions, latency, cost, and error rates. Version prompts like code (`taxonomy_v1.7`, `summary_v2.0`) and tie each deploy to an evaluation report.

By breaking the work into stages, you separate concerns, contain failures, and create clear points where you can measure quality or cost. It is also easier to swap components — for example, replacing the classifier stage with a deterministic model for a high-volume feed — without rebuilding the whole pipeline.

This is what it means to move from prompt engineer to system orchestrator. The prompt is still important, but it is one cog in a machine that must be designed to run safely, repeatedly, and at scale.



LLM-specific design constraints

Designing for LLMs in production is not the same as designing for traditional ML models. The constraints are different, and they shape every decision you make in the system.

Context window limits

Every token you send and receive counts towards the model's context limit. If the

model only supports 16k tokens, you cannot just dump a whole document set into it. Even when models offer million-token windows, more is not always better — noise still hurts performance, and cost scales linearly with size. You need to decide what *must* go in and what can be retrieved on demand.

Temperature and top-p

These parameters control variability in the output. High temperature can be useful for brainstorming, but it introduces instability in production. Low temperature reduces creativity but makes outputs more predictable. The trade-off is not academic (ie, it needs an evaluator), but it is important because it affects consistency and how often you hit guardrails.

Latency

In a playground, a 5-second response might feel fast. In production, it can be unacceptable. If your product requires sub-second responses, you either need smaller models, pre-computation, caching, or a routing layer that avoids calling the LLM for low-impact cases. Latency budgets must be defined before you design the prompt chain.

Cost per call

LLMs are metered services. A verbose prompt or excessive chaining can multiply costs, especially at scale. You should monitor cost per thousand requests alongside

performance metrics, and have a clear plan for what to cut when costs spike without degrading the user experience.

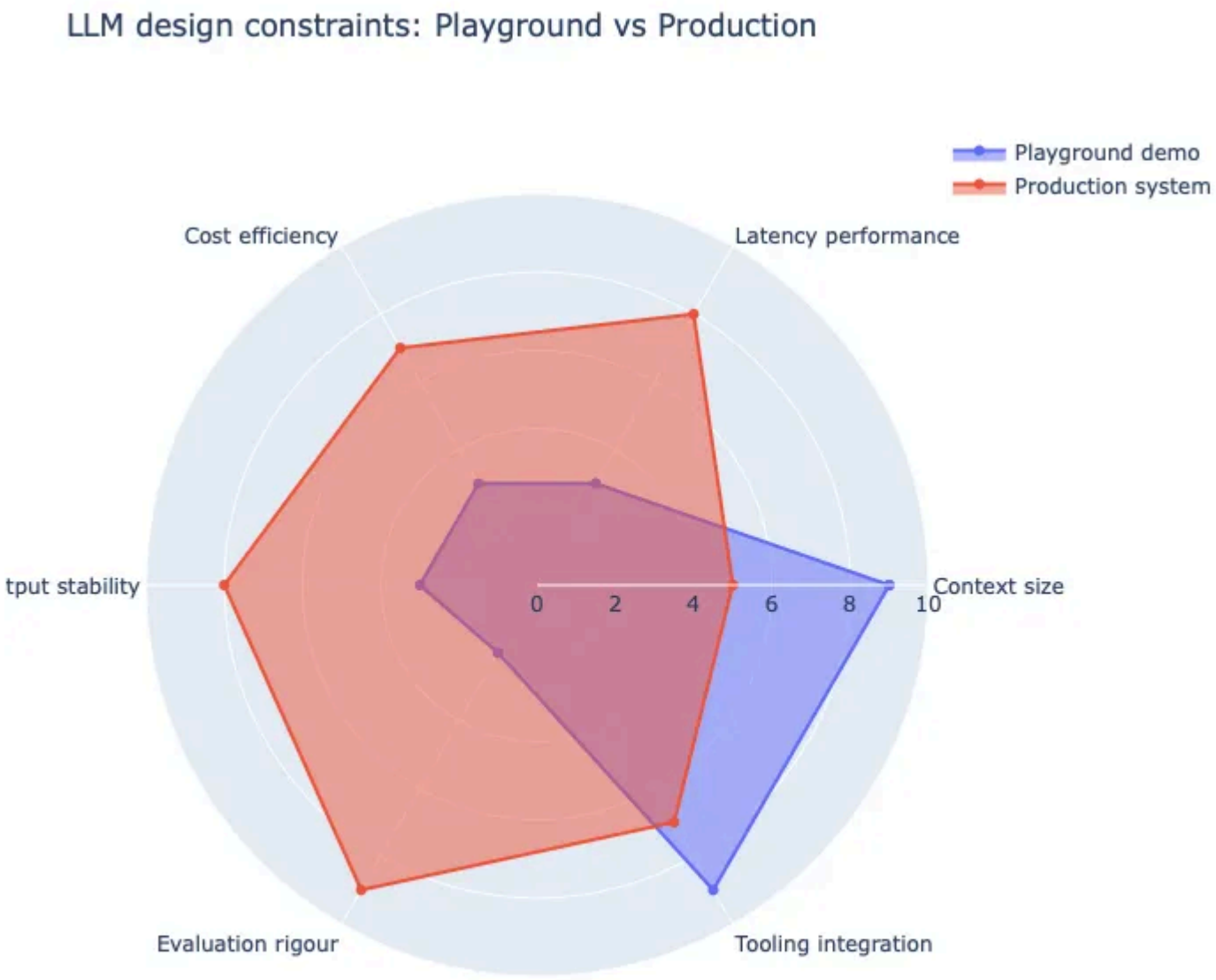
Prompt evaluation

As we previously discussed, I want to reinforce the need of evaluating the quality of generative content. Unlike a model accuracy score, LLM quality is harder to measure automatically. You need a mix of:

- **Golden sets** — manually labelled examples to track precision and recall.
- **LLM-as-evaluator** — a second model scoring the first, with periodic human spot checks.
- **Downstream metrics** — tracking the impact of outputs on user behaviour or business KPIs.

Tool and agent integration

Sometimes the LLM should call an external tool — a search API, a database query, a deterministic parser — instead of trying to generate the answer itself. This reduces hallucination risk, but it adds orchestration complexity and potential failure points.


LLM design constraints: Playground vs Production
Playground demo
Production system
Cost efficiency
Latency performance
tput stability
Context size
0
2
4
6
8
10
Evaluation rigour
Tooling integration

For data scientists, these constraints are not foreign. They are the LLM equivalent of feature limits, training budgets, inference SLAs, and evaluation pipelines. The mental model is the same: know your system's boundaries, and design for them from the start.

Reframing the data scientist's role (for those working purely with GenAI applications)

I have repeated this multiple times: if you have spent time building and deploying machine learning models, you already have most of the instincts needed to design robust LLM systems. The components may be different, but the thinking is the same. For example:

- **Feature selection → prompt context curation**
  In ML, you choose features that give the model the best chance of learning the right patterns. In LLMs, you choose the context that gives the model the best chance of producing a reliable answer. Both require understanding the problem, the data, and the trade-offs.
- **Pipeline building → prompt chaining and output parsing**
  A good ML pipeline sequences data preparation, model training, and evaluation. A good LLM pipeline sequences prompts, tool calls, validation steps, and downstream integrations.
- **Model evaluation → LLM output grading and comparison**
  We grade ML models against metrics like RMSE, ROC-AUC, or precision/recall. For LLMs, we score outputs against golden sets, human ratings, and downstream

KPIs. The methods differ, but the principle of measuring before changing stays the same.

- **Model debugging → hallucination tracing and prompt drift tracking**
  In ML, you check for data leakage, bias, or poor segment performance. In LLMs, you investigate why the model produced an incorrect or unsafe output, track when a prompt starts producing different results, and analyse whether the issue is model-side, context-side, or orchestration-side.

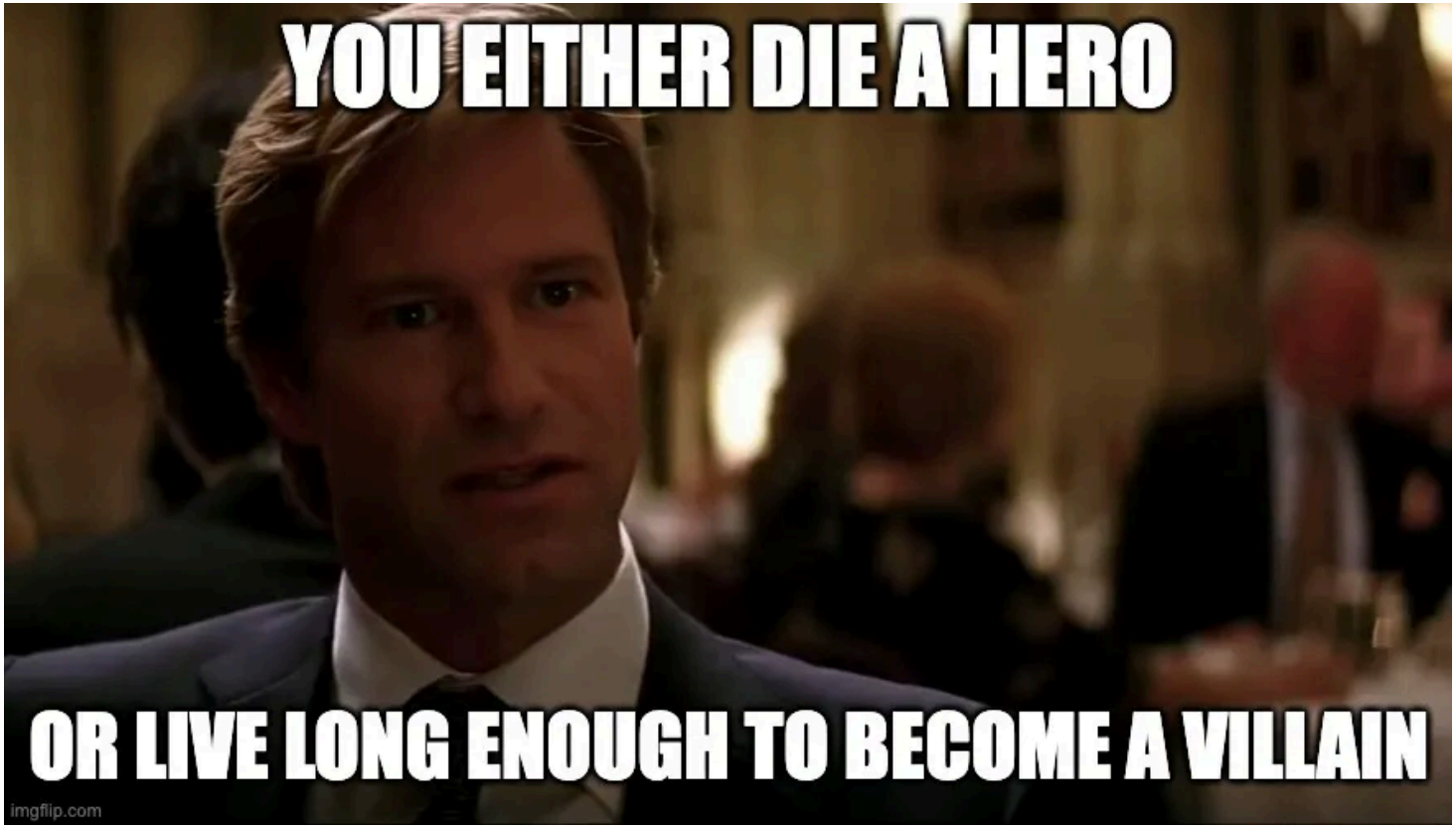
But if you are a data scientist working on classical ML or deep learning systems, your role remains as valid and important as ever.

GenAI is not the new default path for everyone. As we explored in Part 1 and Part 2, much of the world's high-impact work still relies on models that have nothing to do with LLMs, and that will not change anytime soon.

What GenAI changes is that you now have another path available. You can choose to focus on GenAI applications as your primary work, or you can enhance classical ML systems with GenAI components — for example, using an LLM to preprocess messy data before feeding it into a model, or to create hybrid systems where deterministic ML handles precision tasks and the LLM provides reasoning or explanation.

If you do decide to work purely in the GenAI application space, the mapping above is a useful way to think about the transition. You are not starting from scratch. You are

applying the same instincts to a different class of models.



Closing thoughts.

A prompt is not a system. It is an instruction — a fragile one — and it will break the moment the inputs change, the model updates, or the surrounding product shifts.

The uncomfortable truth is that production AI is not magical. It is infrastructure. That is why the skill that will matter most in the next wave of GenAI adoption is not *"being good at writing prompts"*. It will probably be the discipline to design systems that survive contact with reality.

For data scientists, this is both a challenge and a chance. If you work in GenAI applications, **your role is to make it deliver value every day, without surprises.** That means chaining logic, building guardrails, planning fallbacks, and constantly measuring whether it is still doing the job you designed it to do.

If you work in classical ML, **the point is not to abandon it.** Those systems still run the backbone of most high-value products. But you can borrow from the GenAI playbook, and, if you choose, merge the two into hybrid systems that combine precision with adaptability.

The AI hype cycle will pass (as did the Big Data one). The organisations that keep value on the other side will be the ones that treated LLMs as part of a system, not as a party trick. And the data scientists who designed those systems will be the ones still in the room when the spotlight moves on.

Now, I want to hear from you!

- If prompting is just the plumbing, where do you think data scientists can have the biggest impact in building real GenAI systems?
- Have you seen a case where an impressive prompt or prototype failed once it hit production — because the orchestration, guardrails, or monitoring were missing?
- Or a time when designing the system around the LLM completely changed its reliability and value?
- What is your view — will system design become the real differentiator for GenAI applications, or will prompting skills still dominate?

Share your stories, lessons, or counterpoints in the comments. I would love to hear how you are approaching GenAI in your own work 👇

And see you in the next post about *Data Science in the age of AI.* 👋

Further reading

If you are interested in more content, here is an article capturing all my written blogs!

All my articles, one convenient place

JOSE PARREÑO GARCIA • 15 OCTOBER 2024

Read full story →

4 Likes • 4 Restacks

Discussion about this post

Comments Restacks

Write a comment...

© 2026 Jose Parreño Garcia · Privacy · Terms · Collection notice
Substack is the home for great culture